MLB-MAC: Multi-Level Binary MAC Array for Energy Efficient ML Accelerators

Ali Ansarmohammadi, Reza Hojabr, Marzie Mastalizade, Najmeh Nazari, and Mostafa E. Salehi, *Member, IEEE*

***Abstract*— Quantization is a critical compression technique for optimizing deep neural networks (DNNs) on resource-constrained embedded devices. Efficient hardware utilization hinges on effective number representation. Integer representation, a widely adopted method, uses scaling factors and offsets to enhance network accuracy and simplify fractional bit selection through uniform quantization. On the other hand, non-uniform quantization is well-suited for DNNs with parameters following a normal distribution, helping reduce data width requirements. This paper introduces a novel non-uniform representation called MLB (Multi-Level Binary), which encompasses and extends integer representation. We propose an architecture that objectively compares these representations, demonstrating that MLB is a superset of integer representation in terms of accuracy. Our comprehensive analysis spans various data widths, parallel factorization, and DNN models. Our findings indicate that MLB outperforms integer representation in energy efficiency for lower bit widths (2–5 bits), whereas integer representation is more advantageous for higher bit widths (4–8 bits). Specifically, our work shows an average energy improvement of 1.1 to 2.3× and area saving up to 1.7× compared to integer multiply-accumulate (MAC) units, while preserving network accuracy. This research provides insights into the optimal choice of number representation based on bit-width requirements, highlighting the potential of MLB in enhancing the performance and efficiency of DNNs on embedded devices.**

***Index Terms*— DNN, quantization, representation, architecture, energy-efficiency.**

I. Introduction

ARTIFICIAL Intelligence (AI) has achieved unprecedented levels of performance in various real-world applications ranging from image classification, computer vision, and object recognition [1], [2] to diffusion generative models and large language models [3], [4], [5]. As the adoption of emerging AI models grows and computing demands increase, it is imperative to take advantage of the silicon horsepower available in edge devices to achieve additional benefits of performance, privacy, and security [6]. However, most recent AI models are quite large and will not easily fit into edge devices with limited memory and computational resources and tight power consumption constraints.

Extensive research has focused on quantization, a popular compression technique converting activations or weights (32-bit floating-point) into lower bitwidth representations. Quantization spans bitwidths from 16-bit fixed-point to 1-bit representation (aka binarized neural networks). Two main quantization approaches exist: post-training quantization [7], [8], [9] and quantization-aware training [10], [11], [12], [13]. Post-training quantization directly quantizes a model without fine-tuning, offering simplicity and on-the-fly deployment. However, accuracy loss might occur. Quantization-aware training integrates quantization into training, letting the network learn quantization. This method usually offers better accuracy but requires more computational resources and hyperparameter tuning.

Most approaches for low-bitwidth quantization proposed to date have employed uniform quantization where a single step size parameter is configured as the width of the quantization bin [10], [14]. On the other hand, non-uniform quantization methods utilize a non-linear mapping that discretize the representation range into varying step sizes in contrast to uniform methods in which the step size is constant. Non-uniform quantization provides a finer-grained step size which leads to higher representation quality and better network accuracy whereas in uniform quantization, the impact of transitions between the quantized states is ignored as the step size is constant [10]. Despite non-uniform quantization benefits, it requires a more complex hardware design in order to employ non-linear mapping. It also requires a complete rethink of Multiply-and-Accumulate (MAC) unit architecture and memory layout. This work details the hardware requirements of non-uniform quantization and compares them against conventional hardware implementation of uniform quantization methods.

Uniform quantization (hereafter *integer* quantization) employs only integer arithmetic operations on quantized values that require conventional integer multipliers and adders [11], [15]. In an M-bit integer representation system, real values are mapped to a range $[-2^{M-1}, 2^{M-1} - 1]$, where each quantized value is composed of a linear combination of power-of-two bases. On the contrary, [7] proposed a non-uniform quantization scheme where each value is represented as a linear combination of multiple binary weight bases. We refer to this representation system as Multi-Level Binary (MLB) hereafter. In an M-bit MLB representation, a quantized value X_q is represented as $X_q = \alpha_1 b_1 + \alpha_2 b_2 + \ldots + \alpha_M b_M$ where $b_i \in \{0, 1\}$. The bases (α_i) may or may not be a power-of-two, meaning that the values are mapped to a non-uniform range where the distribution pattern can be tuned by adjusting the

Received 9 September 2024; revised 2 December 2024 and 8 March 2025; accepted 30 March 2025. Date of publication 22 May 2025; date of current version 1 October 2025. This article was recommended by Associate Editor X. Fong. *(Corresponding author: Mostafa E. Salehi.)*

The authors are with School of Electrical and Computer Engineering, University of Tehran,Tehran 1417466191, Iran (e-mail: mersali@ut.ac.ir).

Digital Object Identifier 10.1109/TCSI.2025.3562454

bases. In fact, MLB is a superset of the integer representation system; when $\alpha_i = 2^i$, the MLB representation of value X_q will be identical to its standard integer format. As a result, MLB implementation of neural networks can offer the same accuracy as integer models and, with proper quantization-aware training, potentially improves the accuracy even further.

However, when the α_i values in MLB arithmetic operations are not powers of two, the operations differ significantly from traditional integer operations. Multiplication in MLB relies on a bit-wise XNOR-Popcount operation, which enables efficient bit-wise operations and supports parallel processing. On the other hand, the memory requirements in MLB are slightly different, as it is necessary to store both the bases and the binary values on a per-layer basis for activations or a per-channel basis for weights. Additionally, the output of XNOR-Popcount multiplication is an integer, which incurs an extra cost for converting the MAC unit outputs back to MLB format before passing them to the next layer.

This work details the hardware implementation of MLB processing elements, targeting dot-product operations in large neural models. We propose *MLB-MAC*, a novel MLB-based MAC array that offers better energy efficiency and more power/area savings compared to conventional integer MAC arrays in certain bit widths and configurations. We evaluate the accuracy and power/area of *MLB-MAC* using several large DNNs by sweeping the design parameters such as bitwidth and the length of the dot-product. We perform a detailed analysis and show that as the bitwidth lowers, the MLB surpasses the integer implementation in terms of energy while it preserves the network accuracy. In this work, we explore the MLB representation system and its applications in neural network models. Specifically, we achieve the following:

- we prove and demonstrate that MLB is a superset of integer representation in terms of accuracy by showing that an M-bit MLB number can represent each M-bit number in the integer range.
- We propose *MLB-MAC*, a new MAC array based on an MLB representation system for dot-product operations.
- We carry out an in-depth analysis of the proposed MAC array compared to integer counterparts, discuss the impact of design parameters on energy efficiency, area/power savings, memory model, and network accuracy.

We evaluate the RTL implementation of *MLB-MAC* in 45nm technology point targeting five large vision models that are commonly used in edge devices. *MLB-MAC* offers on average,1.1 to 2.3× energy improvement and area saving up to 1.7× compared to integer MACs while preserving the network accuracy. Our findings indicate that MLB outperforms integer representation in energy efficiency for lower bits (2-5), while integer representation is preferable for 4-8 bits.

The remainder of this paper is organized as follows. Section II reviews related work. Section III introduces the background for integers and MLB, and explains our motivation. Section IV presents our proposed architecture for integer and MLB representation. Section V provides a comprehensive analysis of various parameters and networks, along with their results. Finally, Section VI presents the conclusion.

II. Related Work

Quantization techniques are utilized to compress deep neural networks by reducing bit-widths of DNN weights and activations. This leads to more energy-efficient deployment on resource-limited devices while maintaining minimal accuracy loss. The representation system, bit-width, and architecture of the processing elements have a significant impact on the energy efficiency of DNNs.

Integer representation is the most common approach in quantization. Studies [8], [16], and [17] have shown that deploying 16-bit quantization yields accuracy comparable to 32-bit networks. Research like [17] indicates that different networks and layers require varying bit-widths for quantization, with [18] and [19] using quantization-aware training to reduce bit-width to 8 bits. Architectures [20] and [21] focus on pure fixed-point representation to enhance energy efficiency.

Other techniques aim to improve accuracy by mitigating quantization error. References [10] and [12] decrease bit-width below 8 bits by aligning the dynamic range of quantizers during training. However, these methods often neglect hardware overhead from representing weights and activations with extra parameters. Methods like Differentiable Soft Quantization [13] narrow the gap between full-precision and quantized networks but overlook hardware implementation costs. linear output = wieght*activation+bias activation means input

Trainable quantization has been employed by methods like QIL [12] and LSQ [10] to discretize weight and activation parameters. QIL introduces a trainable quantization interval, reducing output task loss and increasing accuracy. LSQ+ [11] refines this approach using offset values for asymmetric quantization. Additionally, uL2Q [22] aims to optimize quantization for normally distributed data, leading to hardware overhead from scaling factor and offset parameters.

Bitpruning [23] employs offsets to align weights and activations within the dynamic range of limited bit representations. Both [22] and [23] multiply fixed-point numbers with scaling factors and add offsets. However, this approach increases the hardware cost due to four partial multiplication components in each parameter multiplication product, compared to using a single fixed-point number.

Binary networks, exemplified by BinaryConnect, achieve high compression rates in memory and computation. BinaryConnect employs binary weights 1, −1, obviating resource-intensive multiplication. BNN [24] extends this by binarizing both weights and activations, replacing multiply-accumulate (MAC) operations with XNOR and Popcount. XNOR-Net [25] presents BWN and XNOR-Net methods, with BWN [26] introducing scaling factors for binary weights, and XNOR-Net applying scaling to both binary weights and activations.

Ternary Weight Network TWN [27] utilizes ternary weights 1, 0, −1 to enhance accuracy and reduce operation complexity. Despite requiring two bits for storage (one more than binary), TWN achieves a 16× compression rate compared to 32 bits.

Works like [7], [28], [29], and [30] address accuracy gaps between binary and full precision networks. They employ linear combinations of multiple binary bits with scaling factors instead of 32-bit floating point numbers. Research shows that using 3 or 4-bit linear combinations for weights and activations

TABLE I
THE ACCURACY OF LOW PRECISION QUANTIZATION METHODS ON IMAGENET

Method	Metrics				Resnet-18			
	Uniform / Non-uniform	Scaling Factor	Offset	Activation Encoder	2-bit	3-bit	4-bit	32-bit
QIL [12]	Uniform	No	No	No	65.7	69.2	70.1	70.2
LSQ [10]	Uniform	Yes	No	No	66.7	69.4	70.7	70.1
LSQ+ [11]	Uniform	Yes	Yes(A)	No	66.8	69.4	70.8	70.1
DSQ [13]	Uniform	Yes	Yes(A,W)	No	65.17	68.66	69.56	70.1
uL2Q [23]	Uniform	Yes	Yes(A,W)	No	65.5	–	66	70.1
LLSQ [37]	Uniform	Yes	No	No	–	68.1	69.8	70.2
CSQ [38]	Uniform	Yes	Yes(A,W)	No	66.9	69.5	70.6	70.6
LQNet [30]	Non-uniform	Yes	No	Yes	64.9	68.2	69.3	70.1
LCQ [39]	Non-uniform	Yes	Yes	Yes	68.9	70.6	71.5	70.4
N2U [40]	Uniform	Yes	No	Yes	69.4	71.9	72.9	71.8
Ours	Non-uniform	Yes	No	Yes	68.9	70.6	71.5	70.6

yields acceptable accuracy close to full precision, yielding benefits in low bit width and bitwise operations.

Several architectural approaches have been proposed to improve area, power, and energy efficiency in accelerators. Many leverage co-design strategies, combining high-level algorithmic optimizations—such as quantization, structured sparsity, and numerical representation—with specialized processing elements to maximize efficiency. One key trend is bit-level sparsity, which enables energy-efficient computation by skipping unnecessary operations, similar to how binary neural networks benefit from bit-wise operations. Serialization techniques further enhance efficiency by reducing the processing overhead of sparse data.

Recent work has focused on bit-sparsity-aware architectures to optimize neural network accelerators [20], [31], [32], [33]. Pragmatic [31] and Bitlet [32] exploit bit sparsity but face challenges in load balancing. BitWave [34] mitigates this by dynamically skipping zero bits at a coarse bit-column granularity, reducing computational overhead in deep neural networks. BitVert [33] extends this approach by dynamically adjusting bit-skipping strategies based on bit distribution. It skips zero bits when they dominate a column and switches to skipping one bits when they are fewer, optimizing computational efficiency. These advancements highlight the potential of bit-level sparsity in improving accelerator designs.

With various quantization techniques demonstrating similar accuracy but distinct hardware overhead, our survey motivates a comparative investigation. We aim to identify the most energy-efficient solution for resource-limited devices by assessing scaling factors, offsets, data types, and bit widths. Table I summarizes quantizers considering these parameters.

III. BACKGROUND AND MOTIVATION

In this section, we comprehensively explain the MLB and integer representations, presenting them as distinct systems with similar syntax. Subsequently, our motivation emerges by introducing a simplified processing unit designed to operate on MLB representations with higher energy efficiency than the integer counterparts.

A. *Multi-Level Binary Representation*

Integer representation, as a form of uniform quantization, transforms full-precision real numbers, denoted as $\mathbf{x}$, into low-precision integer equivalents x^q. These quantized values can be expressed as a linear combination of bits. Precisely, an unsigned integer x^q represented by a M-bit binary encoding can be regarded as the inner product between a scaling factor vector $\boldsymbol{\alpha} = [\alpha_1, \alpha_2, \ldots, \alpha_M]^T$ and the binary coding vector $\mathbf{b} = [b_1, b_2, \ldots, b_M]^T$ where $\alpha_i = 2^i$ *and* $b_i \in 0, 1$. binary as can be 0 or 1

$$x^q = \boldsymbol{\alpha}^T \mathbf{b} \tag{1}$$

As outlined in [7], the scaling factor values, α_i, can be extended to encompass any real numbers to introduce a non-uniform quantization approach. Consequently, Equation 1 can be adapted to include $\alpha_i \in \mathbb{R}$ and $b_i \in \{0, 1\}$ to accommodate unsigned numbers. To handle signed numbers, b_i can be extended to $b_i \in \{-1, 1\}$. We refer to this representation as Multi-Level Binarize (MLB).

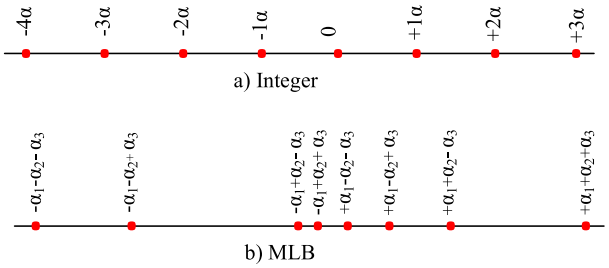

Fig. 1. Integer and MLB representation for 3 bits.

Figure 1 clarifies the difference between these two number representation systems. The figure demonstrates the quantization levels for $M = 3$ in these approaches. In integer representation, the step sizes for quantization levels stay constant; however, in the MLB approach, the step sizes vary. Weight parameters in DNNs follow a normal distribution [14], [22], [29], [35], and multiple research studies [22], [28], [29] suggest that utilizing non-uniform quantization improves the accuracy of mapping weight parameters to quantization levels.

The primary computational complexity in deep neural networks arises from convolution and fully-connected layers, deconstructed into a series of dot-products between activation and weight vectors. A dot-product, denoted as $dot(\mathbf{x}, \mathbf{w})$, accumulates the element-wise products of $\mathbf{x} \in \mathbb{R}^K$, the input activation from the preceding network layer, and $\mathbf{w} \in \mathbb{R}^K$, the weight vector, with K being the dot-product's size. The

substantial computational demands of dot-products and extensive memory access present significant challenges for deep neural networks, especially on resource-constrained devices.

Quantization techniques are employed to alleviate memory storage requirements and expedite the computation of deep neural networks. The objective of the quantization function $Q(\mathbf{x})$ is to represent a floating-point vector $\mathbf{x}$ using fewer bits, thereby approximating $dot(\mathbf{x}, \mathbf{w})$ as $dot(\mathbf{x}^q, \mathbf{w}^q)$, where $\mathbf{x}^q$ and $\mathbf{w}^q$ are the quantized equivalents of $\mathbf{x}$ and $\mathbf{w}$.

Several quantization methods exist, with the integer representation being the most popular. In the context of quantized networks, earlier integer-based methods utilized scaling factors to enhance accuracy [10]. While integer techniques effectively accelerate DNN computations, a noticeable accuracy gap remains between quantized and full-precision models. To tackle this challenge, numerous efforts have focused on minimizing quantization errors by better aligning real values with quantization levels. This has led to incorporating an offset in the number system representation [11].

Initially, let's delve into the computation of dot-products using the Integer representation, which employs a scaling factor and an offset. Following that, we'll explore the dot-product computation utilizing the MLB representation.

B. Scaling Factors and Offset

Based on [11], the scaling factors for weights (α_w) and activations (α_x) and an offset(β) for activations can help to better quantize and fit the data to quantization levels. Considering x_k^q and w_k^q for $k = 1, 2, \ldots, K$ with M-bit quantization and $\alpha_x, \alpha_w, \beta_w \in \mathbb{R}$ as scaling factors and offset for all K elements, x_k^q and w_k^q can be represented as below,

$$w_k^q = \alpha_w w_{dk}, \quad x_k^q = \alpha_x x_{dk} + \beta_x \tag{2}$$

where x_{dk} and w_{dk} are M-bit integers.

The dot-product operation can be calculated as below:

$$\begin{aligned} dot(\mathbf{x}^q, \mathbf{w}^q) &= \mathbf{x}^q[\mathbf{w}^q]^T \\ &= \sum_{k=1}^{K} (\alpha_x x_{dk} + \beta_x) \times (\alpha_w w_{dk}) \\ &= \alpha_x \alpha_w \sum_{k=1}^{K} (x_{dk} w_{dk}) + \underbrace{\beta_x \alpha_w \sum_{k=1}^{K} w_{dk}}_{\beta_{wx}} \\ &= \alpha_x \alpha_w \sum_{k=1}^{K} (x_{dk} w_{dk}) + \beta_{xw} \end{aligned} \tag{3}$$

For each dot-product operation, K M-bit MAC operations are needed. The partial products then will be scaled by $\alpha_x \alpha_w$ scaling factors and finally, the offset β_{xw} will be added.

Multi-Level Binary: Considering x_k^q and w_k^q for $k = 1, 2 \ldots, K$ same as Eq. 1, with M-bit MLB quantization and $\boldsymbol{\alpha}_x, \boldsymbol{\alpha}_w \in \mathbb{R}^M$ as scaling factors for K elements, x_k^q and w_k^q can be represented as below:

$$x_k^q = \sum_{i=1}^{M} \alpha_{x_i} \; s_{x_{k,i}} \; w_k^q = \sum_{j=1}^{M} \alpha_{w_j} \; s_{w_{k,j}} \tag{4}$$

b in eqn1

where $s \in \{-1, 1\}$ symbols are used for binary values. The dot-product operation of two vectors can be demonstrated as the summation of individual bits multiplied by scaling factors:

$$\begin{aligned} &dot(\mathbf{x}^q, \mathbf{w}^q) \\ &= \sum_{k=1}^{K} \left(x_k^q w_k^q\right) \\ &= \sum_{k=1}^{K} \left(\left(\sum_{i=1}^{M} \alpha_{x_i} \; s_{x_{k,i}} \right) \left(\sum_{j=1}^{M} \alpha_{w_j} \; s_{w_{k,j}} \right) \right) \\ &= \sum_{k=1}^{K} \left(\sum_{i=1}^{M} \sum_{j=1}^{M} \alpha_{x_i} \alpha_{w_j} \; s_{x_{k,i}} s_{w_{k,j}} \right) \end{aligned} \tag{5}$$

As mentioned before, $\boldsymbol{\alpha_x}$ and $\boldsymbol{\alpha_w}$ are the same for all K elements of $\mathbf{x^q}$ and $\mathbf{w^q}$, therefore $\alpha_{x_i} \alpha_{w_j}$ can be factorized from Σ_k and after reordering summations, Eq. 5 can be simplified to

$$\sum_{i=1}^{M} \sum_{j=1}^{M} \alpha_{x_i} \alpha_{w_j} \sum_{k=1}^{K} s_{x_{k,i}} s_{w_{k,j}} \tag{6}$$

The Σ_k operator of Eq. 6 can be computed by bitwise xnor_popcount operation based on [24] as below:

$$\sum_{i=1}^{M} \sum_{j=1}^{M} \alpha_{x_i} \alpha_{w_j} xnor\_popcnt\left(b_{x_{k=1..K,i}}, b_{w_{k=1..K,j}}\right) \tag{7}$$

where $b_{x_{k=1..K,i}}, b_{w_{k=1..K,j}}$ are the binary encoding corresponding to $s_{x_{k,i}}, \; s_{w_{k,j}}, \; \forall k \in \{1, \ldots, K\}$.

For example, if $M = 2$ (activations and weight are quantized to 2-bit numbers), four *xnor-popnt* operations are required to calculate the input vectors' dot-product. In this example Eq. 7 would be

$$\begin{aligned} &\alpha_{x_1} \alpha_{w_1} xnor\_popcnt(b_{x_{k=1..K,1}}, b_{w_{k=1..K,1}}) \\ &\quad + \alpha_{x_1} \alpha_{w_2} xnor\_popcnt(b_{x_{k=1..K,1}}, b_{w_{k=1..K,2}}) \\ &\quad + \alpha_{x_2} \alpha_{w_1} xnor\_popcnt(b_{x_{k=1..K,2}}, b_{w_{k=1..K,1}}) \\ &\quad + \alpha_{x_2} \alpha_{w_2} xnor\_popcnt(b_{x_{k=1..K,2}}, b_{w_{k=1..K,2}}) \end{aligned} \tag{8}$$

We can parallelize the *xnor-popcount* terms (called partial product) of Eq. 8 to accelerate the operations.

Xnor-Popcnt Operation: As shown in Fig. 3, compute-intensive MAC operations are replaced by XNOR-popcount operations [25] in BNNs. If we map the binary values of -1 and $+1$ to 0 and 1 in hardware, as described in Equation 7, the XNOR operation is first applied between the corresponding weights and activations. The next step is to count how many of the results are 1, a process known as popcount, or denoted as P. Finally, for the dot product result, the value $2P - K$ should be computed. It is important to note that if the binary values are represented by 1 and 0, instead of using the XNOR operation, an AND operation should be used. In this case, the dot product result is simply equal to P

C. Which Quantization Method Is More Energy Efficient at Close Accuracy?

Table I presents the accuracy of different low-precision quantization methods for Resnet-18 model trained on ImageNet, undergo quantization using various methods, encompassing both uniform and non-uniform techniques. Accuracy measurements are reported for quantization levels ranging from 1 to 8 bits. A comparative analysis of the accuracy of quantization methods against the baseline (full-precision) reveals that both 4-bit uniform and non-uniform methods achieve accuracy levels nearly equivalent to full-precision.

Various quantization methods employ different representations, which can lead to differences in their hardware implementations. This diversity in hardware implementation raises questions regarding the energy efficiency of varying quantization methods. Specifically, we are interested in determining whether the energy efficiency across various quantization methods is consistent or if there are notable differences. If there are differences, we aim to identify which quantization method is most energy-efficient.

These inquiries motivated us to undertake a comparative analysis of the energy efficiency of quantization methods. We acknowledge the challenge of conducting a fair comparison between non-uniform and uniform quantization techniques in hardware due to their distinct architectures. Nevertheless, we endeavored to assimilate architectural details as closely as possible for a meaningful comparison.

To the best of our knowledge, no prior studies have specifically addressed comparing different quantization methods in terms of hardware requirements. This comparison is particularly crucial for embedded systems and other resource-limited devices. Given that many quantization methods achieve comparable accuracy levels, the findings from such a comparison could help in selecting the most energy-efficient quantization method for practical implementations.

D. What Is the Impact of Quantization Accuracy Enhancement Techniques on Hardware?

Table I illustrates various hardware parameters that can affect the energy efficiency of each quantization method. The first metric examines uniform and non-uniform representation. Uniform representations typically rely on conventional integer arithmetic, leveraging optimized circuits for computation, while non-uniform representations often require bespoke arithmetic designs tailored to specific circuits. In the case of MLB, exploiting light xnor-popcnt operations could potentially streamline computations, although empirical references supporting this claim are currently lacking.

The second hardware metric considers memory overhead for each method. Factors such as scaling coefficients and offset values can increase memory access, subsequently raising energy consumption during DDR access.

Lastly, the third metric evaluates the need for an encoder to convert activations for preparation in the subsequent layer. For instance, converting final integer activations to MLB format for input into the next layer is necessary in MLB operations.

Parallel parameters are another crucial factor influencing the architecture and ultimately the energy efficiency of quantization methods. Figure 2 depicts a simplified architecture for Integer and MLB representations. When considering only the dot-product operation, the size of the dot-product, the number of bits for activations and weights, and the parallel factor for employing parallel operations are key parameters affecting the architecture.

In MLB, bit-wise operations such as xnor-popcnt come into play, which can significantly impact the design choices related to parallelism. We delve into a comprehensive investigation of these parameters in Section IV.

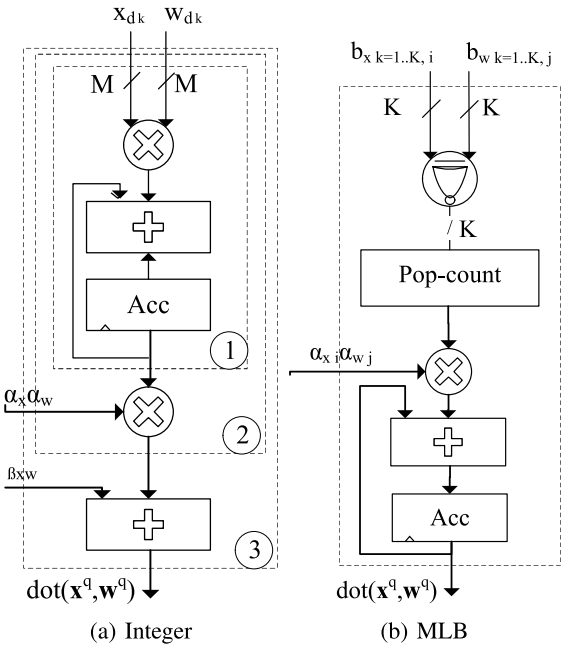

Fig. 2. Basic architecture for (a) Integer and (b) MLB, 1) Normal Integer, 2) Integer with a scaling factor, 3) Integer with the scaling factor and offset.

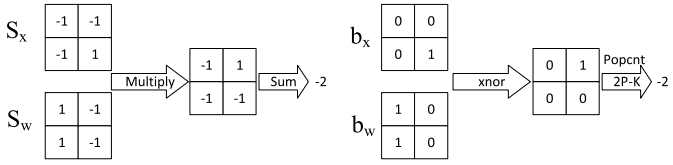

Fig. 3. The relationship between the dot product (left) and xnor-Popcount (right).

These diverse metrics can significantly influence the design and architecture of different quantization methods, particularly non-uniform representations.

This prompts us to develop and implement an architecture for non-uniform representation based on MLB, followed by a comparative analysis against integer-based methods.

IV. MLB Representation and Comparable Architecture

While numerous specific and optimized methods exist for designing and developing architectures for both Integer and MLB representations, our goal is to establish a fair comparison. We aim to create a dot-product operation architecture that is simple yet equitable for both representations. We maintain an equivalent operation volume to ensure fairness by basing our comparison on dot-product computation.

Additionally, memory bandwidth is a crucial consideration in any design involving access to costly off-chip memory. Therefore, we maintain uniformity in both architectures by utilizing the same data width for weights, activations, and identical parallelism factors. This approach ensures a balanced comparison between Integer and MLB representations.

In the subsequent sections, we will present the architecture for the Integer representation. A detailed breakdown of its components and operations will be provided. Subsequently, we will delve into the introduction of MLB as an extension

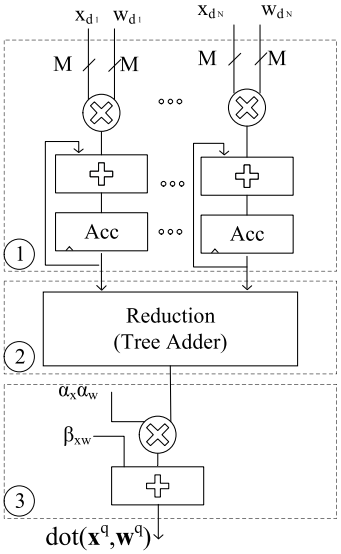

Fig. 4. Integer architecture: 1) Mac unit, 2) Reduce module, 3) Scaling and offset.

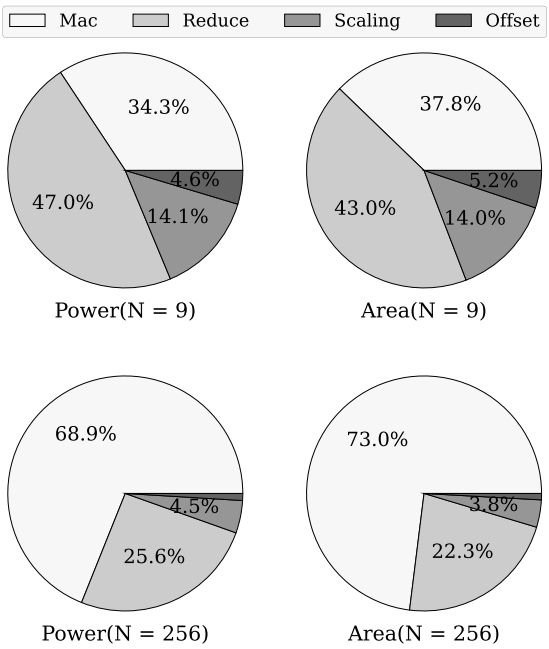

Fig. 5. Power/Area break down for integer reduction.

of Integer representation. Once we establish its foundation, we will present a proposed architecture for MLB. The central component of this architecture is the xnor-popcnt block, and we will offer comprehensive insight into its structure. Following that, we will elaborate on the other elements comprising the architecture for MLB.

We will establish a 2D systolic array for the entire network to complete the comparison. This comparison will be facilitated using the same specifications for both representations within the context of this array.

A. Integer Architecture

Figure 4 displays how Eq. (3) is implemented in hardware. This setup performs dot-product calculations for K-pairs of M-bit activations and weights. To expedite these operations, our design incorporates N parallel MACs. The design is divided into three main sections: a MAC unit, a reduction module, and the scaling and alignment of the final result. The reduction module is implemented using a basic tree adder, which efficiently combines partial sums from the first stage of computation. We utilize the Design Compiler's default synthesis settings to optimize its implementation. This ensures a robust and standardized design for the reduction operation in both architectures

During each cycle, the hardware can carry out multiplication and accumulation for N sets of x_{dk}, w_{dk}, $k \in 1, \ldots, K$, and then aggregate the outcomes in $Acci$. After $\frac{K}{N}$ cycles, these results are directed to the reduction stage, where they are combined with the outputs of other MACs. Ultimately, the aggregated result must be scaled and aligned to generate the final output.

We have introduced clock gating for the reduction and scaling phases to conserve energy, as they only need to run once for all dot-product computations.

Cost of Scaling Factor and Offset: As mentioned in Section I, enhancing accuracy in the Integer method involves incorporating a scaling factor and offset. However, these parameters come with minimal hardware costs. Figure 5 shows the breakdown of each component within the reduction and scaling segment of the architecture. This breakdown underscores the expense associated with the scaling factor, and the offset is small, particularly as N increases. Therefore, utilizing these parameters to enhance the quantization error in the network is advisable.

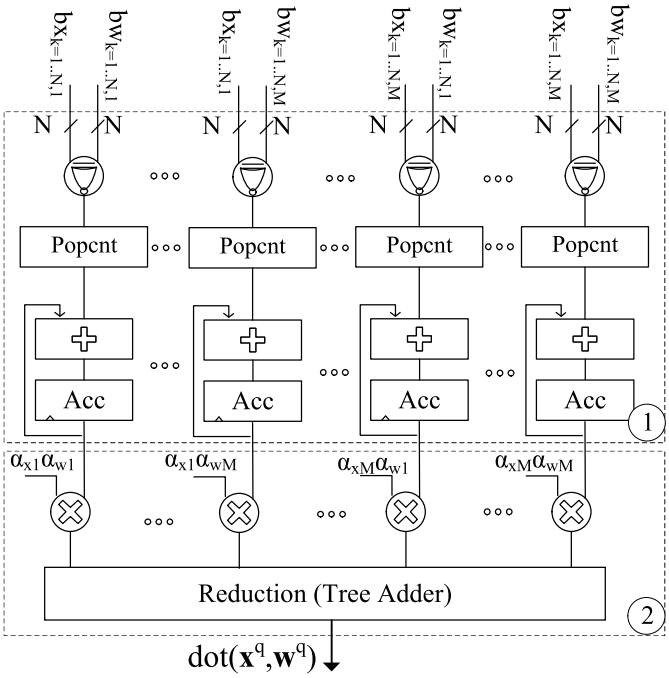

Fig. 6. MLB architecture: 1) xnor-popcnt and accumulator, 2) scaling factor and reduce module.

B. MLB Architecture

Figure 6 depicts the architecture for MLB representation, corresponding to Eq.6. Like the Integer architecture, this

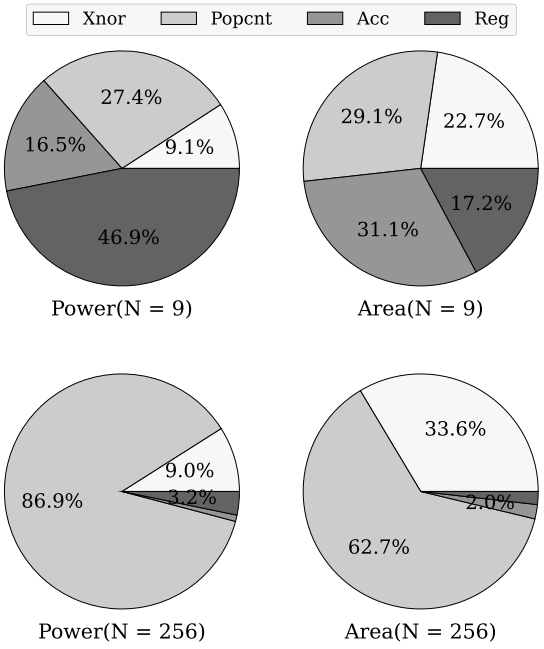

Fig. 7. Power/Area break down for Xnor-popcnt and accumulator.

design employs N to achieve a parallelism factor with a data width of M bits. In general, the architecture comprises three main parts:

xnor-Popcnt and Accumulator: This initial part operates every cycle for N inputs. Figure 7 illustrates the breakdown of the xnor-Popcnt circuit and accumulator. It consists of four parts: xnor, Popcnt, accumulator, and register. As N increases, the Popcount portion becomes more dominant.

Basis Multiplier and Accumulation: The second part involves the basis multiplier and its accumulation.

Activation Encoding: The final stage is an encoder that converts the output into an MLB representation for the next layer. Between the second stage and the activation encoding, an additional layer, such as batch normalization, may be included. The implementation of the activation encoding unit depends on the selected quantization method for the MLB representation. In this work, we adopted the approach proposed in [29] due to its superior accuracy. Following the methodology of [29], activation encoding is implemented using a comparator and a lookup table (LUT) to determine threshold values. These thresholds are used to assign the nearest quantization level to each input.

A detailed breakdown of these three stages will be provided in Section V.

C. MLB: Integer's Comprehensive Superset

This section establishes that MLB serves as a superset for Integer. Any integer value, which can be incorporated using a scaling factor, can be expressed as follows:

$$x_{Int} = \alpha_x \left(-2^{M-1} b_{M-1} + \sum_{m=0}^{M-2} b_m 2^m \right) \quad b_m \in 0, 1 \quad (9)$$

To present these numbers using MLB representation, we can select the scaling factor for MLB as follows:

$$v_{M-1} = -\alpha_x 2^{M-1},$$
$$v_m = \alpha_x 2^m \quad for\ m \in \{0, 1, M-2\} \quad (10)$$

where $b_m(MLB) = b_m(Integer)$. This way, any number within the Integer domain can be represented by an MLB number while maintaining the same data width.

V. Experimental Results

This section offers an all-encompassing presentation of results substantiating a balanced comparison between MLB and integer (Int) architectures. Our approach begins with detailing the methodology employed to derive these results. Subsequently, we initiate the examination by showcasing the power and area breakdown of MLB and Int per operation, along with their respective ratios.

In the following analysis stage, we deeply understand the absolute energy consumption per operation. We present the exact energy per operation and investigate the consequences of amortizing the second-stage components within the MLB and Int architectures. This investigation comprehensively seeks to understand the efficiency mechanisms within these architectures.

To ensure the breadth of our comparison, we extend our examination to encompass the entirety of the network architecture. This comprehensive approach is crucial in revealing MLB and Int architectures' holistic performance and behavior.

Concluding our exploration, we unveil the trade-off between bit width, accuracy, and energy efficiency for each architecture. This final facet offers valuable insights into the delicate balance between these critical parameters within the context of MLB and Int architectures.

A. Experimental Setup and Parameter Selection (or Methodology)

Setup: Architectures were implemented, tested, and verified in Verilog. A standard synthesis tool was utilized for synthesis and performance analysis, producing precise power consumption estimates, area utilization, and timing characteristics. The physical design targeted the 45nm technology node using the FreePDK45 library, offering a good balance between performance and power efficiency. A clock frequency of 250MHz was used for experiments and evaluations because it offered a reasonable trade-off between performance and power usage under real-time operational conditions. It also met all critical paths in our design.

As stated in Section IV, we employed a range of diverse parameters to ensure a fair comparison between Integer and MLB architectures, aligning with the actual size of DNN networks. To provide a comprehensive overview of these parameters and to discuss their applicability across actual DNN layers, we present all the relevant details in Table II.

Data Width (M): As elaborated in section IV, since MLB is superset of integer, it is possible to represent any M-bit integer value combined with scaling factor using an equivalent M-bit value in the MLB representation, maintaining the same

TABLE II
PARAMETER DEFINITION AND ACCEPTED VALUES IN VARIOUS LAYERS

Parameters definition

Name	Explanation
M	Number of bits (data width)
K	Dot product size
N	Hardware parallelism factor(Mac Array Size)

Parameter values in different layer types

Layer type	Acceptable value for K
Conv2D	{2x2, 3x3, ...} × {number of input channel}
DepthWise	{3x3, 5x5, 7x7, ..} × 1
PointWise	{1x1} × {number of input channel}
Dense	Number of input neuron
LSTM	Number of input features + Number of hidden layers
Transformer	...

bit width and employing a specific value for the basis. Consequently, if we quantize a network using the Integer method and achieve a certain accuracy level, we can subsequently convert the quantized value representation to MLB while retaining the same accuracy for the network.

With this approach, we can effectively compare the hardware-related results (energy, power, and area) of the Integer and MLB architectures operating under the same data-width setting. It is important to note that while MLB holds the potential for greater accuracy, we have deferred its exploration to future investigations. The remaining results are compared on equal grounds, considering the same data width for both Integer and MLB approaches.

MAC Array Size(N) and Dot Product Size(K): As mentioned in Section IV, parameter "K" is dot product size. The parameter "N" represents the Parallelism factor, which denotes the number of weights and activations that can enter each clock. Its relationship to "K" is crucial in determining the system's efficiency.

If "K" is less than "N" ($K < N$), it indicates that the system is underutilizing the available parallel processing units. On the other hand, if "K" is greater than "N" ($K > N$), the system needs to reuse the first stage with an accumulator to handle the additional tasks that exceed the available parallel processing units.

The value of "N" is determined based on a specific application's required throughput or processing capacity. However, the target throughput is inherently related to the value of "K". The system's efficiency lies in finding an appropriate balance between "N" and "K" to achieve the desired throughput without wasting resources or causing bottlenecks.

Conclusion: For a comprehensive comparison, we extensively investigate various types of DNN layers and compute their respective dot product sizes, presented in Table II. All the subsequent results are based on these parameters.

Different Layers: DNNs utilize different layer types to achieve optimal performance (accuracy), and these layers have varying dot product sizes, so we aim to ensure a fair and comprehensive comparison. Initially, we compare the MLB and Int architectures using the Mac array granularity. Subsequently, we extend the results to the entire network based on a 2D systolic array [40], as a popular central unit architecture for DNNs. This approach allows us to observe the impact of different parameters in the network domain, as the entire network and its layers should be implemented on a unified hardware configuration set on the systolic array.

B. Design Space Exploration and Trade-Offs for Multiply-Accumulate Arrays

Figures 8 and 9 present a detailed analysis of power and area distribution per operation within the MLB architecture, which is normalized against the Integer architecture. In the architecture outlined in Section IV, the MLB component is divided into three distinct parts. The first part involves XNOR-popcount operations and an accumulator. The second part is the reduction process, encompassing a scaling factor multiplier and reduction operations. Lastly, the third part consists of an encoder quantizing activations for compatibility with the MLB representation. As mentioned, the first part should be performed every cycle, but the second and third parts are required only once for each dot product computation. Our study focuses on understanding the implications of altering bit widths (M), diverse levels of parallelism for MAC arrays (N), and different values for dot product size (K). Subsequently, we thoroughly analyze the effects and attributes linked to the M and N parameters while considering a scenario where K equals N. Following that, we delve into an in-depth exploration of the dot product's size and the specific ratio of K/N.

Impact of Incrementing M on MLB Performance (K==N) As demonstrated in Fig. 9, a notable trend emerges when considering the parameter M's influence on the MLB method's performance. Specifically, an increase in the M value results in a noteworthy consequence concerning power consumption compared to the Integer method for a given N value.

When M is elevated, the power consumption of MLB surpasses that of the Integer method. This phenomenon can be attributed to the intricate interplay of various factors within the architecture. Notably, in the first part, as M increases, the number of parallel components(xnor-popccnt + Acc) experiences a quadratic rise. In contrast, within the Integer multiplier for two M-bit inputs, the output data width increases linearly with 2×M bits. Consequently, as M escalates, the Integer architecture retains an advantageous position in terms of power efficiency. For instance, when we set M to 1 or 2, the power of 2 is either equal to or less than 2 times M. In the context of Fig. 9, when N is 256, the power efficiency of MLB improves significantly by a factor of 4× and 2×, respectively. However, if we increase M to 3, the components in the first part of MLB surge by 9 times. In comparison, the output width of the integer multiplier only grows by 2 × 3=6, as illustrated in Fig. 9. Consequently, the power consumption of MLB surpasses that of the integer multiplier when N is less than 32.

Furthermore, an interesting point is how the Reduce part becomes more significant as M increases. The number of multipliers and the scale of reduction operations surge exponentially, driven by the quadratic growth of M. This intricate scaling underscores the growing computational demand within the Reduce part.

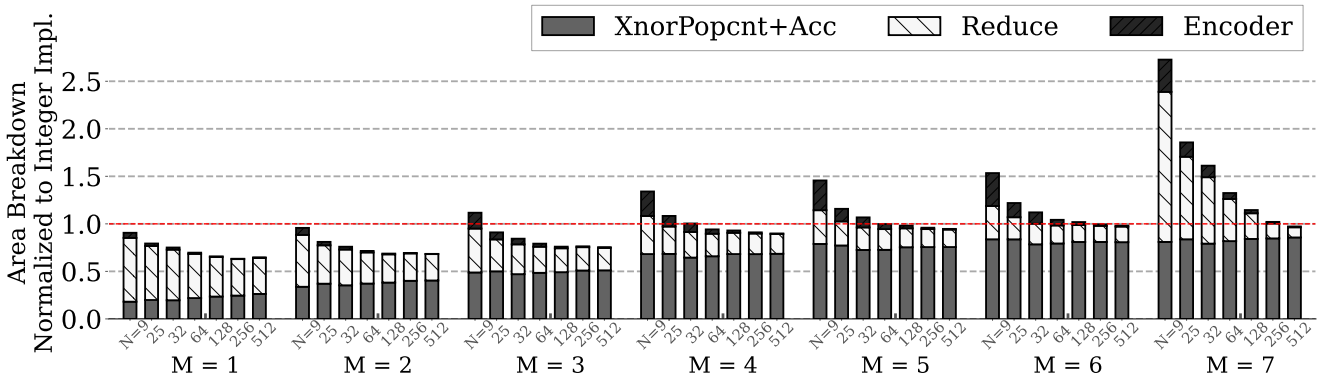

Fig. 8. Breakdown of area/op of *MLB-MAC* normalized to integer implementation.

In the broader context, as M is augmented, the Encoder segment expands, albeit linearly, in tandem with the other components. However, it's noteworthy that this expansion is limited to a factor of M, given the requirement to quantize the output to an M-bit representation.

In conclusion, the dynamics of increasing M within the MLB method reveal intricate trade-offs. While power consumption surges due to the quadratic rise in parallel components, the corresponding growth in the Reduction segment and limited expansion in the Encoder segment contribute to the nuanced behavior observed. Careful consideration of these factors is crucial in determining the optimal value of M for maximizing performance while minimizing power consumption.

Impact of Increasing N on MLB Performance (K=N): Another critical factor affecting the power of both the MLB and Integer methods is N. When we choose a specific M value, increasing N has different effects on these methods.

For the Integer method, as N goes up, its power increases because the number of components working in parallel grows linearly with N. However, for MLB, the situation is more interesting. As N increases, MLB's power becomes more favorable.

Although the Popcnt component's size grows as N increases in MLB, its output doesn't increase linearly with N. Instead, it grows by Log(N), which means the input to the accumulator also grows more slowly. On the other hand, in the Integer method, the number of parallel components in the first part (including the multiplier and accumulator) increases linearly with N, leading to higher power consumption.

One important thing to note is that MLB's Reduction and Encoder parts don't change much with N. But in the Integer method, the Reduction part grows linearly with N, causing an increase in power consumption.

In simpler terms, when we have M set to 5 and N goes beyond 64, MLB becomes more power-efficient than the Integer method. This shows how MLB benefits from larger N values.

In summary, MLB's unique architecture makes it perform better as N grows. At the same time, the Integer method's power increases due to the linear growth in parallel components and the impact on its Reduction part.

Area Breakdown: The breakdown of area, as depicted in Fig. 8, follows the pattern established by the power breakdown. Notably, the ratio of MLB to Int is lower in the area breakdown. This divergence might stem from the prevalence of high signal activity within the MLB architecture, contributing to the elevated power consumption per operation. This observation underscores the potential influence of signaling activity on the increased operational power within the MLB architecture.

Comparing Energy Per Operation and K/N Impact on MLB Performance: In our result's representation, shown in Fig. 10, we illustrate the energy expended per operation across various configurations of M, N, and K. Each row in the Fig. 10 corresponds to a distinct data width and each column represents a different value of N. This investigation covers MLB and Int architectures, providing a comprehensive view of the outcomes. Moving forward, we delve into a detailed analysis of the key insights brought to light by these findings.

1- Amortizing the Reduce Part: When we examine the absolute energy per operation as N increases, a significant trend emerges: an elevated N leads to reduced energy consumption per operation. This reduction can be attributed to the efficient amortization of the reduction cost. This consistent effect is observed in both architectural scenarios.

The energy per operation diminishes as N is increased due to the effective distribution of the reduction cost across a more significant number of inputs. Notably, as the parameter M experiences an increment, the reduction cost for the MLB architecture escalates exponentially with the square of M. Consequently, when N is heightened, the amortization of the reduction cost becomes notably more influential. A concrete illustration of this phenomenon can be found in Fig. 10, particularly for the case of M=6. In instances where N has lower values, the energy expended per operation in the MLB architecture surpasses that of Int. However, a remarkable observation surfaces: as the value of N is progressively increased, the energy per operation for MLB experiences a substantial reduction, eventually falling even below that of the Int architecture.

2- Effect of Incrementing K/N and the Existence of an Optimal K/N: In the MLB architecture context, the multiplier's cost is amortized. Yet, a notable consideration arises when K is substantially increased, given the heightened

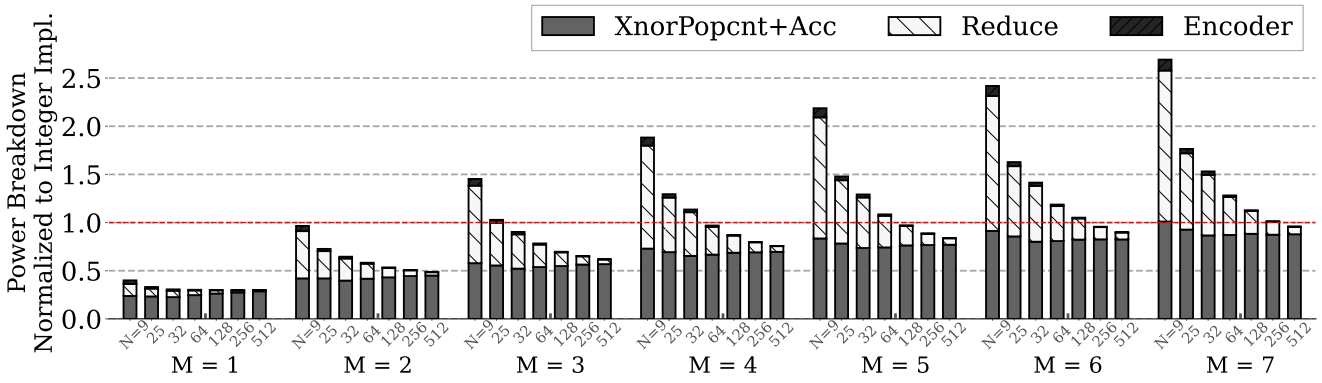

Fig. 9. Breakdown of power/op of *MLB-MAC* normalized to integer implementation.

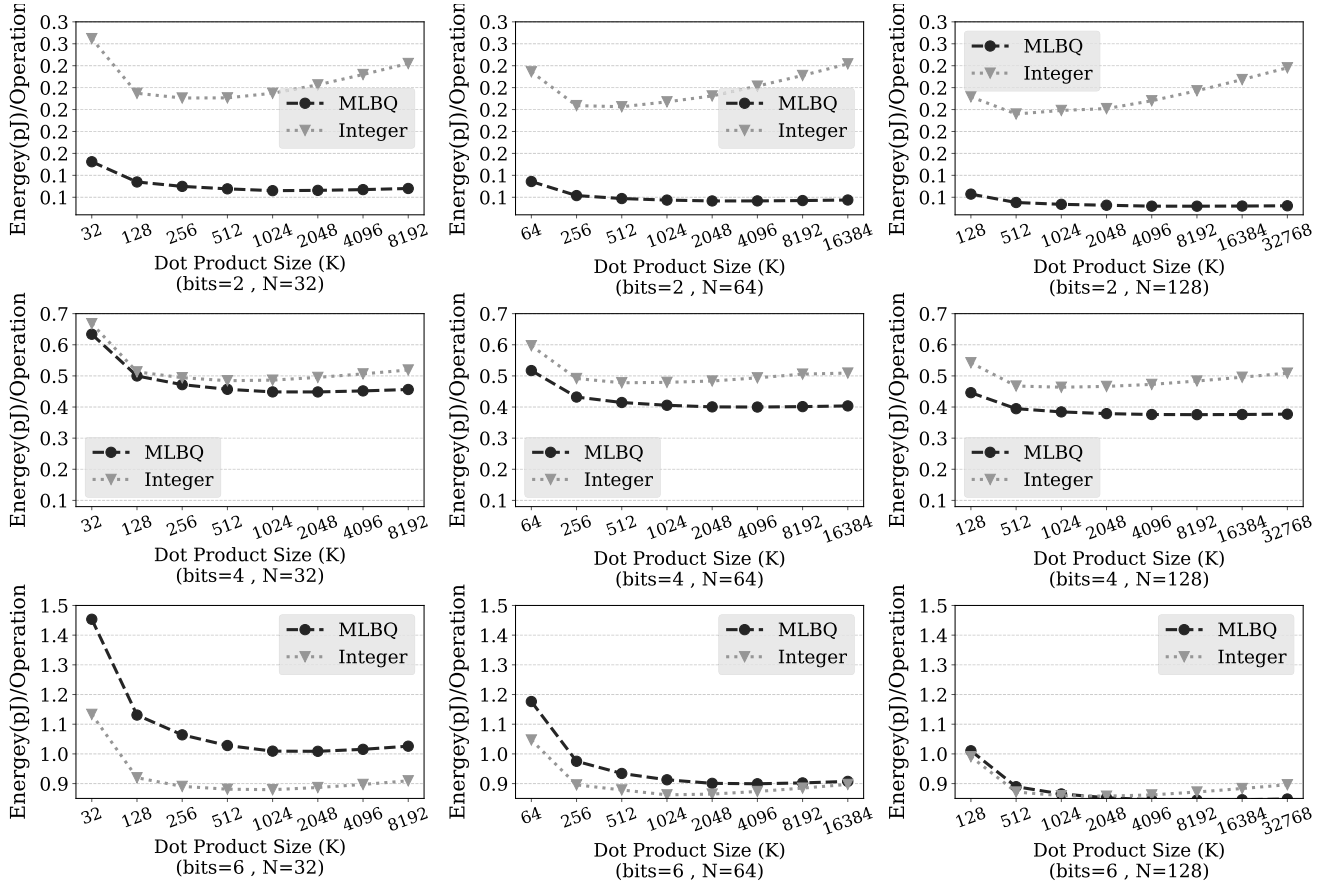

Fig. 10. Energy per operation for different Ms and Ns (with M and N representing data width and parallelism factor, respectively).

expense of the multiplier operation. On the other hand, in the case of the Int architecture, this consideration becomes more pronounced as the K/N ratio increases.

As demonstrated in Fig. 9 and Fig. 8, the trend reveals that when M is increased, the Int architecture outperforms MLB. However, it's worth highlighting that the MLB architecture benefits significantly from an elevated K/N ratio. For instance, in scenarios where M equals 6 and N equals 128, the effectiveness of MLB becomes prominent when the dot product size (K) exceeds 2048. This underscores the role of the K/N ratio in enhancing the performance of MLB architecture.

3- Bit Cost and the Trade-off Between Bit Width and Energy Consumption: Another notable observation in Fig. 10 pertains to the energy cost of each additional bit. Specifically, considering the energy per operation for the MLB architecture, the values for M=2, 4, and 6 are 1.3, 0.65, and 1.45, respectively. In comparison, for the Integer architecture, the respective values are 0.28, 0.68, and 1.15. This analysis provides valuable insights into each architecture's energy efficiency in different bit widths.

C. Results for DNN Based on 2D Systolic Array Architecture

Building on the insights presented in the preceding section, it becomes evident that both N and K play pivotal roles in determining the energy expended per operation. This insight is

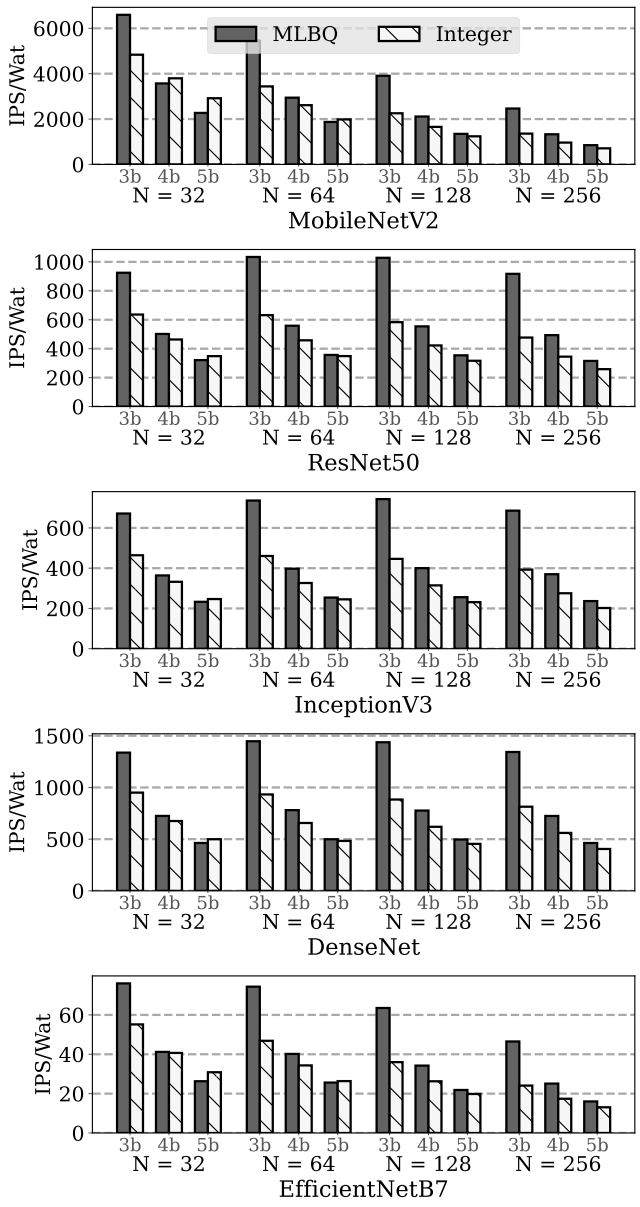

Fig. 11. Comparing Inference/s/Wat (IPS/Wat) MLB and integer for different DNNs and values for M and N.

further corroborated by the findings outlined in Table II, where K exhibits variability across the entire network. In light of this, the subsequent section aims to present results for a state-of-the-art network architecture, encompassing Convolutional, Depthwise, Pointwise, and Dense layers.

The visual representation in Fig.11 provides a comprehensive overview of results across networks, each characterized by varying M and N. Each row in Fig.11 corresponds to a distinct network configuration. The following discussion delves into key points highlighted by the figures.

A Smaller Mac-Array Is Needed for DepthWise Layers: As illustrated in Fig.11, an exciting observation arises concerning the network architectures MobileNet and EfficientNetB7. These architectures do not exhibit an optimal value of N, as increasing N leads to a decrease in ISP/Wat. This phenomenon can be attributed to the fact that MobileNet and EfficientNetB7 encompass numerous DepthWise layers with smaller dot product sizes. Consequently, augmenting N does not contribute to ISP/Wat; instead, it introduces overhead that further diminishes MAC-Array efficiency.

Conversely, a distinct trend emerges for other network architectures, where optimal values of N can be identified. N=64 and 128 are the optimal choices for achieving the highest ISP/Wat efficiency. This observation holds true for most of the networks under consideration. These specific N values are of particular significance in embedded devices with constrained resources. In such environments, achieving optimal ISP/Wat is paramount, and the preference for N=64 and 128 underscores their crucial role in maximizing efficiency within these network architectures.

Bit Cost for IPS/Watt; Comparison of MLB and Int: Our analysis encompasses results across all network architectures for varying bit widths of 3, 4, and 5 bits. This presentation allows for a straightforward observation of the incremental cost associated with each additional bit.

A notable trend in the disparity between MLB and Int regarding IPS/Wat becomes apparent. This difference is particularly pronounced in the context of lower bit widths. However, as the bit width increases, this variance diminishes. Notably, a noteworthy exception occurs with N=32 and M=4, 5, where there are instances when Int outperforms MLB in terms of performance per watt. Nevertheless, the broader trend remains consistent: while performance per watt fluctuates in certain scenarios, MLB generally demonstrates superior performance per watt compared to Int.

Trade-Off Analysis: Accuracy vs. Bit Width vs. Energy Efficiency: To comprehend the trade-off between accuracy and bit width, while considering energy efficiency, refer to Fig. 12. The depicted observations pertain to several renowned DNNs. The data is collected based on the optimal N values, as illustrated in Figure 11. Accuracy is confined within a 5 percent tolerance, contingent on the relative bit width. As referenced in [10], [11], and [41], all accuracy values are culled from cutting-edge quantization research papers.

Pareto-optimal points are indicated on the graph. For instance, if targeting an accuracy of 75.3, Net1 with a 3-bit MLB configuration showcases superior IPS/Watt. Conversely, aiming for an IPS/Watt of 3000, the network with the highest accuracy can be chosen, such as Net2 with 4-bit Int or Net3 with 5-bit MLB.

Memory Overhead and Summarized Area Results: To ensure fair comparison, Table III provides a concise overview of total weights and relative memory overhead for scaling factors in both MLB and Int across different networks and data widths.

Notably, MobileNetv2, being a smaller network, exhibits higher memory overhead. This is mainly due to its smaller total weight volume and numerous DepthWise layers with minimal channel and dot product sizes and higher data width.

On the other hand, in networks like ResNet18 and ResNet50, the memory overhead is relatively reasonable and can be considered negligible.

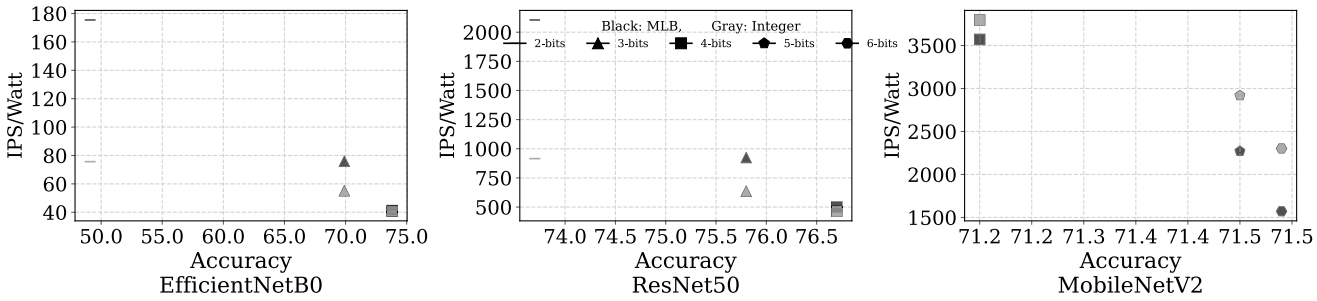

Fig. 12. Accuracy vs. efficiency across different Ms and networks: the horizontal axis represents accuracy. Markers indicate the number of bits, Gray markers represent Integer results, while black markers represent MLB results.

TABLE III
MEMORY OVERHEAD FOR MLB AND INT ARCHITECTURES

Network Name	M	Weights (MByte)	+MLB Mem Over-head(%)	+Int Mem Over-head(%)
EfficientNetB0	2	16.5	11.1	2.8
	3	24.7	16.7	1.9
	4	32.0	22.2	1.4
ResNet18	2	3.6	1.3	0.3
	3	5.5	1.9	0.2
	4	7.3	2.5	0.1
ResNet50	2	9.5	3.4	0.8
	3	12.7	5.2	0.6
	4	15.9	6.9	0.4
MobileNetV2	4	1.1	49.8	3.1
	5	1.4	62.2	2.5
	6	1.6	74.8	2.1

D. Comparison With State of the Art (SoTA)

To provide a fair comparison, we selected BitVert [33] as a baseline, implemented it, and extracted results for direct evaluation against our *MLB-MAC* architecture.

In the BitVert design, the processing element (PE) is configured for an 8-bit data width with a block size of 16 activation-weight pairs. It employs serialization and sparsity techniques to optimize performance. In contrast, our architecture supports variable bit-widths with configurable parallelism, where the block size can be adjusted based on the design (N).

Given that BitVert primarily processes activation data, we extended its PE to support a wider range of data widths. To ensure consistency with BitVert, we also considered the constant values used within its PE, which are associated with static and dynamic sparsity levels (e.g., 10%). We adjusted the constant input data width to 80% of the original data width to account for these sparsity conditions.

To facilitate a fair comparison across various parallelism levels, we scaled the BitVert PE instances to match the processing configurations of our architecture. This scaling ensured iso-bandwidth conditions, allowing for an apples-to-apples evaluation of the two architectures.

The original BitVert implementation was developed using the TSMC 28nm process, while our architecture was designed using FreePDK 45nm. We used the BitVert PE synthesized in FreePDK 45nm as the baseline for all subsequent comparisons.

The comparison results, based on different data widths (2-8 bits) and block sizes (N=16, 64), are summarized in the Table IV. The results indicate that when a higher parallelism factor (N) is used with lower data widths, *MLB-MAC* achieves better energy efficiency (operations/s/watt). We have observed similar results in Section V when comparing MLB and Integer architectures. Higher block sizes (N) in lower data widths, leads to better results in *MLB-MAC*.

TABLE IV
COMPRESION WITH STOA: OPERATION/S PER WATT FOR DIFFERENT DATA WIDTHS (2-8 bits) AND BLOCK SIZES (N=16, 64)

	Operation/s/Watt Normalized to BitVert			
	N=16		N=64	
M	BitVert	*MLB-MAC*	BitVert	*MLB-MAC*
2	1	0.91	1	1.37
3	1	0.77	1	1.20
4	1	0.67	1	1.07
5	1	0.66	1	1.06
6	1	0.64	1	1.03
7	1	0.63	1	1.01
8	1	0.60	1	0.98

VI. CONCLUSION

In conclusion, we have delved into the popular uniform quantization approach represented by Integer, which utilizes scaling factors and offsets. Additionally, we introduced MLB as a non-uniform representation. To ensure a comprehensive and unbiased evaluation, we designed architectures for both Integer and MLB and thoroughly investigated network parameters such as data-width, dot product size, and parallelism factor. Our study provides a detailed hardware comparison of these architectures.

We conducted extensive experiments, examining both dot-product operations in a MAC array and entire network computations using a 2D systolic array. The results firmly indicate that MLB excels in lower bit ranges (1-5 bits), while Integer proves superior for higher bits (4-8 bits). This research not only sheds light on the optimal representation for specific bit-widths but also underscores the significance of comprehensive hardware evaluation when making such comparisons.

VII. ACKNOWLEDGMENT

Reza Hojabr's contribution to this work was made while affiliated with the University of Tehran.

REFERENCES

[1] C. Szegedy, A. Toshev, and D. Erhan, "Deep neural networks for object detection," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 26, Dec. 2013, pp. 2553–2561.

[2] M. Nixon and A. Aguado, *Feature Extraction and Image Processing for Computer Vision*. New York, NY, USA: Academic, 2019.
[3] J. Wei et al., "Emergent abilities of large language models," 2022, *arXiv:2206.07682*.
[4] L. Floridi and M. Chiriatti, "GPT-3: Its nature, scope, limits, and consequences," *Minds Mach.*, vol. 30, no. 4, pp. 681–694, Dec. 2020.
[5] H. Touvron et al., "LLaMA: Open and efficient foundation language models," 2023, *arXiv:2302.13971*.
[6] Qualcomm Technologies, Inc. (2023). *The Future of Ai is Hybrid*. [Online]. Available: https://www.qualcomm.com/news/onq/2023/05/how-on-device-ai-is-enabling-generative-ai-to-scale
[7] X. Lin, C. Zhao, and W. Pan, "Towards accurate binary convolutional neural network," in *Proc. Adv. Neural Inf. Process. Syst.*, Jan. 2017, pp. 345–353.
[8] Y. Chen et al., "DaDianNao: A machine-learning supercomputer," in *Proc. 47th Annu. IEEE/ACM Int. Symp. Microarchitecture*, Dec. 2014, pp. 609–622.
[9] T. Chen et al., "DianNao: A small-footprint high-throughput accelerator for ubiquitous machine-learning," *ACM SIGARCH Comput. Archit. News*, vol. 42, no. 1, pp. 269–284, 2014.
[10] S. K. Esser, J. L. McKinstry, D. Bablani, R. Appuswamy, and D. S. Modha, "Learned step size quantization," 2019, *arXiv:1902.08153*.
[11] Y. Bhalgat, J. Lee, M. Nagel, T. Blankevoort, and N. Kwak, "LSQ+: Improving low-bit quantization through learnable offsets and better initialization," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, Jun. 2020, pp. 696–697.
[12] S. Jung et al., "Learning to quantize deep networks by optimizing quantization intervals with task loss," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 4350–4359.
[13] R. Gong et al., "Differentiable soft quantization: Bridging full-precision and low-bit neural networks," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 4852–4861.
[14] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding," 2015, *arXiv:1510.00149*.
[15] B. Jacob et al., "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 2704–2713.
[16] M. Courbariaux, Y. Bengio, and J.-P. David, "Training deep neural networks with low precision multiplications," 2014, *arXiv:1412.7024*.
[17] S. Gupta, A. Agrawal, K. Gopalakrishnan, and P. Narayanan, "Deep learning with limited numerical precision," in *Proc. Int. Conf. Mach. Learn.*, 2015, pp. 1737–1746.
[18] P. Gysel, J. Pimentel, M. Motamedi, and S. Ghiasi, "Ristretto: A framework for empirical study of resource-efficient inference in convolutional neural networks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 29, no. 11, pp. 5784–5789, Nov. 2018.
[19] P. Gysel, M. Motamedi, and S. Ghiasi, "Hardware-oriented approximation of convolutional neural networks," 2016, *arXiv:1604.03168*.
[20] S. Sharify et al., "Laconic deep learning inference acceleration," in *Proc. ACM/IEEE 46th Annu. Int. Symp. Comput. Archit. (ISCA)*, Jun. 2019, pp. 304–317.
[21] S. Ghodrati, H. Sharma, C. Young, N. S. Kim, and H. Esmaeilzadeh, "Bit-parallel vector composability for neural acceleration," in *Proc. 57th ACM/IEEE Design Autom. Conf. (DAC)*, Jul. 2020, pp. 1–6.
[22] C. Gong et al., "μL2Q: An ultra-low loss quantization method for DNN compression," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Jul. 2019, pp. 1–8.
[23] M. Nikolić et al., "BitPruning: Learning bitlengths for aggressive and accurate quantization," 2020, *arXiv:2002.03090*.
[24] M. Courbariaux, I. Hubara, D. Soudry, R. El-Yaniv, and Y. Bengio, "Binarized neural networks: Training deep neural networks with weights and activations constrained to +1 or -1," 2016, *arXiv:1602.02830*.
[25] M. Rastegari, V. Ordonez, J. Redmon, and A. Farhadi, "XNOR-Net: ImageNet classification using binary convolutional neural networks," in *Proc. Eur. Conf. Comput. Vis.* Cham, Switzerland: Springer, 2016, pp. 525–542.
[26] M. Courbariaux, Y. Bengio, and J. David, "BinaryConnect: Training deep neural networks with binary weights during propagations," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 28, Dec. 2015, pp. 3123–3131.
[27] F. Li, B. Liu, X. Wang, B. Zhang, and J. Yan, "Ternary weight networks," 2016, *arXiv:1605.04711*.
[28] M. Ghasemzadeh, M. Samragh, and F. Koushanfar, "Rebnet: Residual binarized neural network," in *Proc. IEEE 26th Annu. Int. Symp. Field-Program. Custom Comput. Mach. (FCCM)*, Jun. 2018, pp. 57–64.
[29] D. Zhang, J. Yang, D. Ye, and G. Hua, "LQ-Nets: Learned quantization for highly accurate and compact deep neural networks," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2018, pp. 365–382.
[30] I. Chung et al., "Extremely low bit transformer quantization for on-device neural machine translation," 2020, *arXiv:2009.07453*.
[31] J. Albericio et al., "Bit-pragmatic deep neural network computing," in *Proc. 50th Annu. IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, Oct. 2017, pp. 382–394.
[32] H. Lu et al., "Distilling bit-level sparsity parallelism for general purpose deep learning acceleration," in *Proc. 54th Annu. IEEE/ACM Int. Symp. Microarchit.*, Oct. 2021, pp. 963–976.
[33] Y. Chen, J. Meng, J.-S. Seo, and M. S. Abdelfattah, "BBS: Bi-directional bit-level sparsity for deep learning acceleration," in *Proc. 57th IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, 2024, pp. 551–564.
[34] M. Shi, V. Jain, A. Joseph, M. Meijer, and M. Verhelst, "BitWave: Exploiting column-based bit-level sparsity for deep learning acceleration," in *Proc. IEEE Int. Symp. High-Performance Comput. Archit. (HPCA)*, Mar. 2024, pp. 732–746.
[35] N. Abramson, D. Braverman, and G. Sebestyen, "Pattern recognition and machine learning," *Springer Google schola*, vol. 9, no. 4, pp. 257–261, Oct. 1963.
[36] X. Zhao, Y. Wang, X. Cai, C. Liu, and L. Zhang, "Linear symmetric quantization of neural networks for low-precision integer hardware," in *Proc. Int. Conf. Learn. Represent.*, 2020, pp. 1–16.
[37] F. Asim, J. Park, A. Azamat, and J. Lee, "Centered symmetric quantization for hardware-efficient low-bit neural networks," in *Proc. BMVC*, 2022, p. 538.
[38] K. Yamamoto, "Learnable companding quantization for accurate low-bit neural networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 5029–5038.
[39] Z. Liu, K.-T. Cheng, D. Huang, E. Xing, and Z. Shen, "Nonuniform-to-uniform quantization: Towards accurate quantization via generalized straight-through estimation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 4942–4952.
[40] N. Jouppi et al., "In-datacenter performance analysis of a tensor processing unit," in *Proc. 44th Annu. Int. Symp. Comput. Architect.*, 2017, pp. 1–12.
[41] S.-T. Lin, Z. Li, Y.-H. Cheng, H.-W. Kuo, C.-C. Lu, and K.-T. Tang, "LG-LSQ: Learned gradient linear symmetric quantization," 2022, *arXiv:2202.09009*.

Ali Ansarmohammadi received the B.Sc. and M.Sc. degrees in computer engineering from the University of Tehran, Iran, in 2012 and 2015, respectively, where he is currently pursuing the Ph.D. degree in system and computer architecture. His research interests include computer architecture, embedded system design, hardware accelerators, and hardware/software co-design for machine learning algorithms.

Reza Hojabr received the B.Sc. degree in computer engineering from the K. N. Toosi University of Technology, Tehran, Iran, in 2012, and the M.Sc. and Ph.D. degrees in computer science from the University of Tehran, Tehran, in 2014 and 2020, respectively. From 2019 to 2021, he was a Ph.D. Researcher with Simon Fraser University, Burnaby, BC, Canada. His research interests include computer architecture, AI hardware accelerators, and hardware/software co-design for machine learning algorithms.

Marzie Mastalizade received the B.Sc. and M.Sc. degrees in computer engineering, with a focus on computer architecture from the University of Tehran, in 2019 and 2023, respectively. Her research interests include deep neural networks, embedded systems, and image processing, with a current emphasis on designing energy-efficient architectures for resource-constrained platforms.

Najmeh Nazari received the B.Sc. degree in computer engineering from Shiraz University in 2010 and the M.Sc. degree in computer engineering from Isfahan University of Technology in 2013. She is currently pursuing the Ph.D. degree. From 2013 to 2015, she was a Lecturer with the Shahid Chamran University, Ahvaz. Her research interests include deep learning, embedded systems, computer architecture, and hardware security.

Mostafa E. Salehi (Member, IEEE) received the B.Sc. degree in computer engineering from the University of Tehran, Iran, in 2001, the M.Sc. degree in computer engineering from the Amirkabir University of Technology in 2003, and the Ph.D. degree in computer engineering from the University of Tehran in 2010. He is currently an Assistant Professor with the School of Electrical and Computer Engineering, University of Tehran. His current research interests include computer architecture, embedded system design, hardware/software co-design, and design of neural network accelerator.